

Information Retrieval 2

Machine Learning and Information Retrieval

Makoto P. Kato, National Institute of Informatics / University of Tsukuba

This lecture is based on the following book:

Jimmy Lin, Rodrigo Nogueira, Andrew Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Springer, 2021.

Document ranking = Ordering documents by estimated relevance to a query. The core technology of search

BM25 = A scoring function integrating TF saturation + document length normalization + IDF. Still the standard method for first-stage retrieval

Inverted index = A term-to-document reverse lookup structure. Enables fast search even over large-scale collections

Vocabulary mismatch problem = When a query and document express the same concept with different words, exact matching fails to retrieve them

Building on the limitations of BM25, this lecture covers **Learning to Rank** and **Neural IR**

BM25 is powerful, but a single scoring function has its limits. We will learn about the progression from machine learning that integrates multiple signals to deep learning that eliminates the burden of feature engineering

First Half: Learning to Rank

Limitations of BM25 and the need for machine learning

Feature engineering

Three categories of Learning to Rank (Pointwise / Pairwise / Listwise)

Derivation of RankNet's loss function and gradients

LambdaRank / LambdaMART

Second Half: The Dawn of Neural IR

Changes deep learning brings to IR

Representation-based models (DSSM)

Interaction-based models (DRMM, KNRM)

The arrival of BERT and its impact

Multi-stage ranking architecture

Learning to Rank

BM25 is a scoring function based solely on **term frequency** and **document length**. However, in real-world search, there are many additional signals for judging document relevance.

Vocabulary mismatch: "automobile" and "car" refer to the same concept, but BM25 does not match them

Lack of compound signals: Cannot incorporate non-textual information such as page popularity, link structure, or click history

Ignoring context: BM25 is Bag-of-Words and does not consider word order or semantic relationships

Parameter limitations: Only two free parameters, k_1 and b . Little room for tuning

Examples of signals BM25 cannot capture

Whether query terms appear in the title

PageRank (authority based on link structure)

Domain trustworthiness (.gov, .edu, etc.)

Past click-through rates (behavioral history for the same query)

Document freshness (last modified date)

Fundamental problem: BM25 is a single hand-crafted scoring function and has no mechanism to integrate these diverse signals. A **framework that automatically combines multiple signals** is needed

Learning to Rank (LTR) is an approach that automatically acquires a ranking function through **supervised learning** based on hand-designed **features**.

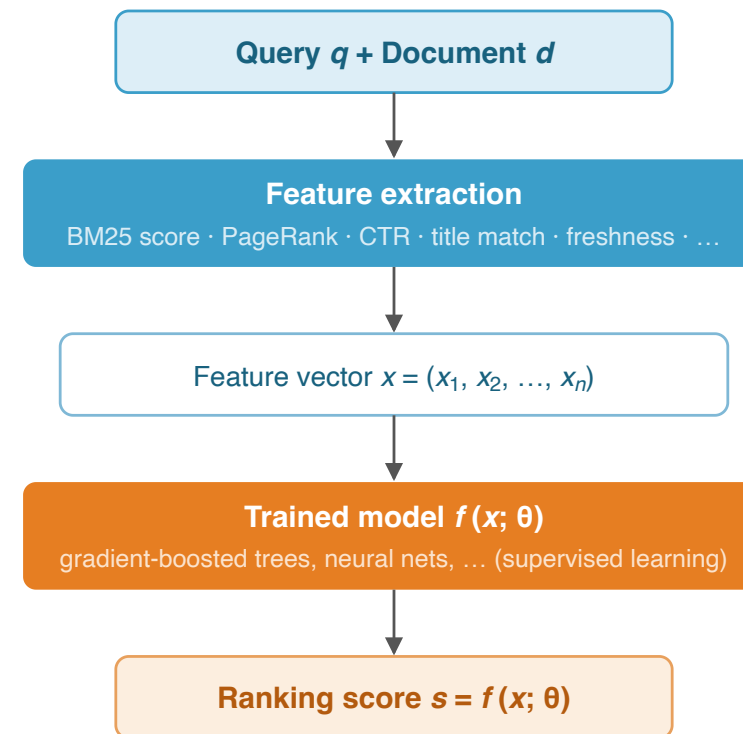
Basic procedure:

1. Extract a **feature vector** $\mathbf{x} = (x_1, x_2, \dots, x_n)$ from each query q and document d pair
2. Train model $f(\mathbf{x}; \theta)$ using training data (query-document pairs annotated with relevance labels)
3. At search time, compute scores from each document's features and rank by score

Research began in the late 1980s [Fuhr, 1989]

Reached its peak in the early 2010s

Real-world search engines use **hundreds of features or more**



Training data: query–document pairs annotated with relevance labels

Real-world search engines combine hundreds of features

— BM25 is integrated as one of them, not replaced

In Learning to Rank, **diverse features** are extracted from query-document pairs. Below are 6 representative categories. Real-world search engines combine **hundreds of features**.

1. Text Statistics

BM25 score (per title/body/anchor), TF, IDF, document length

2. Proximity Features

Co-occurrence frequency of query term pairs (within 5 words), co-occurrence in title/anchor

3. Text-Specific

JavaScript volume, HTML tag ratio, spam score, readability score

4. Web Features

PageRank, inlink count, domain authority, URL depth

5. Query Features

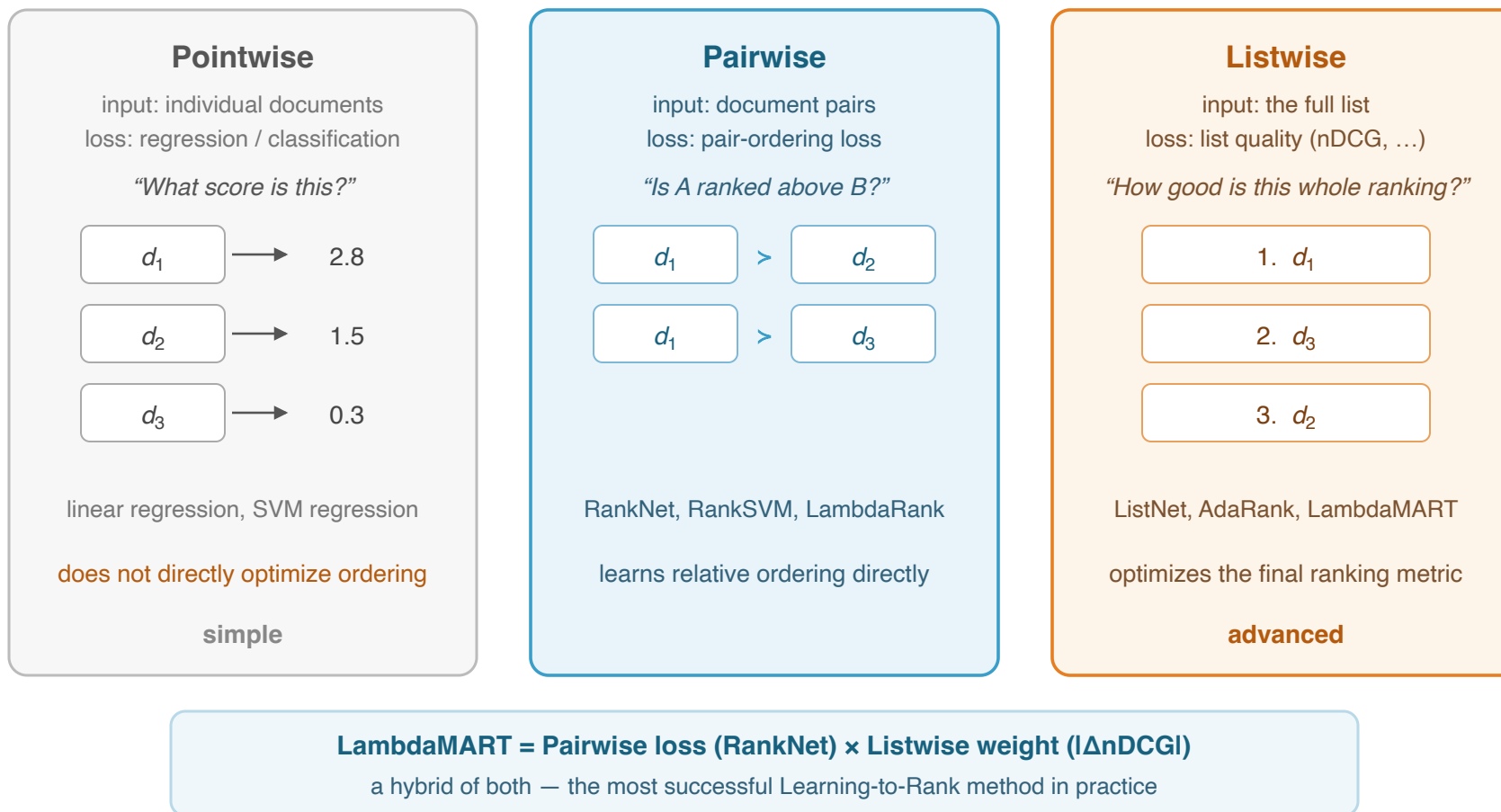
Query length, query frequency, query type (navigational/informational)

6. User Behavior

Click-through rate (CTR), dwell time, bounce rate ->
The most powerful signal

Note: BM25 scores themselves are used as one of the features in LTR. BM25 is not "replaced" but rather "integrated"

Depending on what unit the loss function is computed over, LTR is classified into **three approaches**.



Intuitive difference: Pointwise asks "What is this document's score?", Pairwise asks "Is A or B ranked higher?", Listwise asks "How good is this entire ranking?"

Pointwise converts ranking into a standard **regression** or **classification** problem.

Pointwise Formulation

Predict relevance y from features $\mathbf{x}$ of a query-document pair:

$$f(\mathbf{x}; \theta) \rightarrow y \quad L = \sum_i \ell(f(\mathbf{x}_i; \theta), y_i)$$

Regression: $\ell = (f(\mathbf{x}_i) - y_i)^2$, Classification: $\ell = -y_i \log f(\mathbf{x}_i)$

Existing regression/classification algorithms can be used directly

Strengths and weaknesses:

Strengths

- Simple, easy to implement
- Existing ML libraries usable directly

Weaknesses

- Does not directly optimize **relative ordering** between documents
- The essence of ranking is "order," but Pointwise optimizes absolute score values
- Comparison information between documents for the same query is lost

Let us illustrate what it means that Pointwise "ignores ordering" with a concrete example.

Setup: Three documents for query "Kyoto autumn leaves"

Document	True relevance	Predicted score
A: Kyoto autumn foliage guide	3 (highly relevant)	2.8
B: Arashiyama tourism guide	2 (moderately relevant)	2.9
C: Tokyo cherry blossom info	0 (not relevant)	0.2

Pointwise loss (squared error):

$$L_A = (2.8 - 3)^2 = 0.04$$

$$L_B = (2.9 - 2)^2 = 0.81$$

$$L_C = (0.2 - 0)^2 = 0.04$$

$$\text{Total: } L = 0.89$$

What is the problem?

Predicted ranking: $B(2.9) > A(2.8) > C(0.2)$

Correct ranking: $A(3) > B(2) > C(0)$

The most important ordering "A above B" is reversed, but in Pointwise loss, this only shows up as score error for Document B (0.81).

If the predictions were $A=2.1$, $B=2.0$, $C=0.5$:

Total loss: $0.81 + 0 + 0.25 = 1.06$ (increased!)

But the ranking $A > B > C$ is **correct**

Lesson: Pointwise optimizes "score accuracy," but what matters in ranking is "ordering correctness."
This gap motivates the Pairwise approach.

Pairwise learns the **relative ordering** of document pairs. The goal is to capture the preference that "document d_i is more relevant than document d_j ."

RankNet [Burges et al., 2005] is a representative method of this approach, developed at Microsoft Research.

Basic idea:

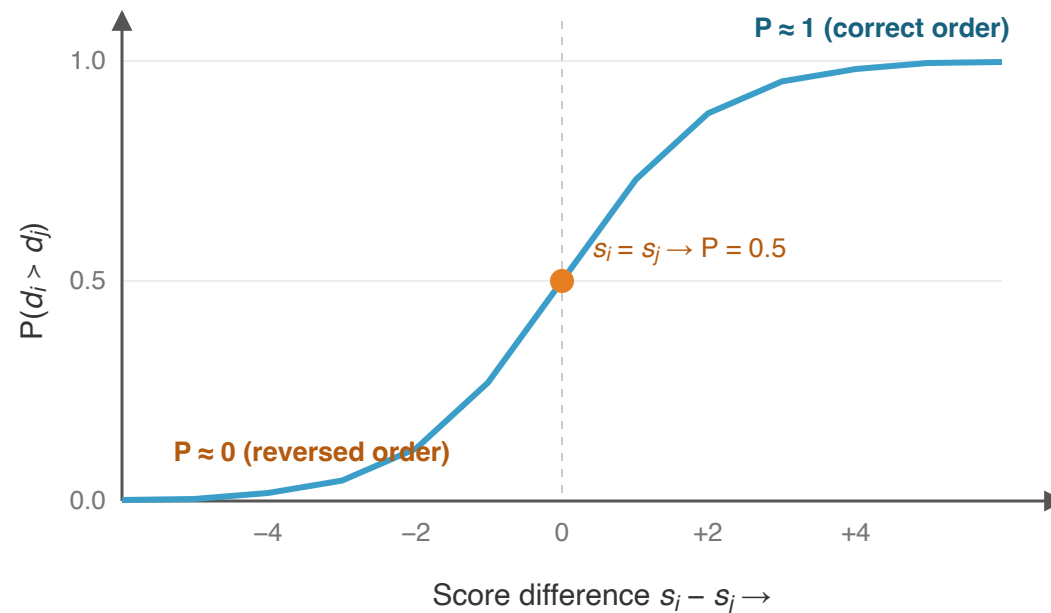
The model outputs scores $s_i = f(\mathbf{x}_i; \theta)$, $s_j = f(\mathbf{x}_j; \theta)$

Model the **probability** that d_i is ranked above d_j from the score difference $s_i - s_j$

Using the sigmoid function:

$$P(d_i \succ d_j) = \frac{1}{1 + e^{-\alpha(s_i - s_j)}}$$

$\alpha > 0$: Scaling parameter (steepness of the curve)



$\alpha > 0$ controls the steepness of the curve (Burges et al., ICML 2005)

Difference from Pointwise: Only the score **difference**, not absolute values, determines the ordering probability. $s_i = 10, s_j = 8$ and $s_i = 3, s_j = 1$ yield the same ordering probability

RankNet uses cross-entropy loss to minimize the divergence between the predicted ordering probability and the ground-truth ordering probability.

Step 1: Defining the ground-truth labels

Define the "ground-truth ordering probability" $\bar{P}_{ij}$ from the true labels:

$$\bar{P}_{ij} = \begin{cases} 1 & (\text{if } d_i \text{ is more relevant than } d_j) \\ 0.5 & (\text{if equal}) \\ 0 & (\text{if } d_j \text{ is more relevant than } d_i) \end{cases}$$

Step 2: Model's predicted probability (restated from previous slide)

$$P_{ij} = P(d_i \succ d_j) = \frac{1}{1 + e^{-\alpha(s_i - s_j)}}$$

Step 3: Cross-entropy loss

Cross-entropy between the ground-truth distribution $\bar{P}_{ij}$ and the predicted distribution P_{ij} :

$$L = -\bar{P}_{ij} \log P_{ij} - (1 - \bar{P}_{ij}) \log(1 - P_{ij})$$

This has the same structure as binary classification cross-entropy. It measures the divergence between the ground-truth and prediction regarding the probability that " d_i should rank higher."

The most practically important case: derive the loss when d_i is more relevant than d_j ($\bar{P}_{ij} = 1$).

$$\begin{aligned} L &= - \underbrace{1}_{\bar{P}_{ij}} \cdot \log P_{ij} - \underbrace{(1-1)}_{1-\bar{P}_{ij}} \cdot \log(1 - P_{ij}) \\ &= -\log P_{ij} \quad (\text{the second term vanishes}) \end{aligned}$$

Substituting $P_{ij} = \frac{1}{1+e^{-\alpha(s_i-s_j)}}$:

$$L = -\log \frac{1}{1 + e^{-\alpha(s_i-s_j)}} = \log(1 + e^{-\alpha(s_i-s_j)})$$

RankNet loss function (when d_i is more relevant than d_j)

$$L_{ij} = \log(1 + e^{-\alpha(s_i-s_j)})$$

$s_i > s_j$ (correct order) $\rightarrow L \rightarrow 0$ (small loss)

$s_i < s_j$ (incorrect order) $\rightarrow L$ is large (penalty)

$s_i = s_j \rightarrow L = \log 2 \approx 0.693$

Let $\alpha = 1$ and compute the loss with concrete numbers.

Case 1: Correct order ($s_i = 3, s_j = 1$)

$$\begin{aligned} L &= \log(1 + e^{-1 \cdot (3-1)}) = \log(1 + e^{-2}) \\ &= \log(1 + 0.135) = \log(1.135) = 0.127 \end{aligned}$$

Small loss -> The model has learned the correct order

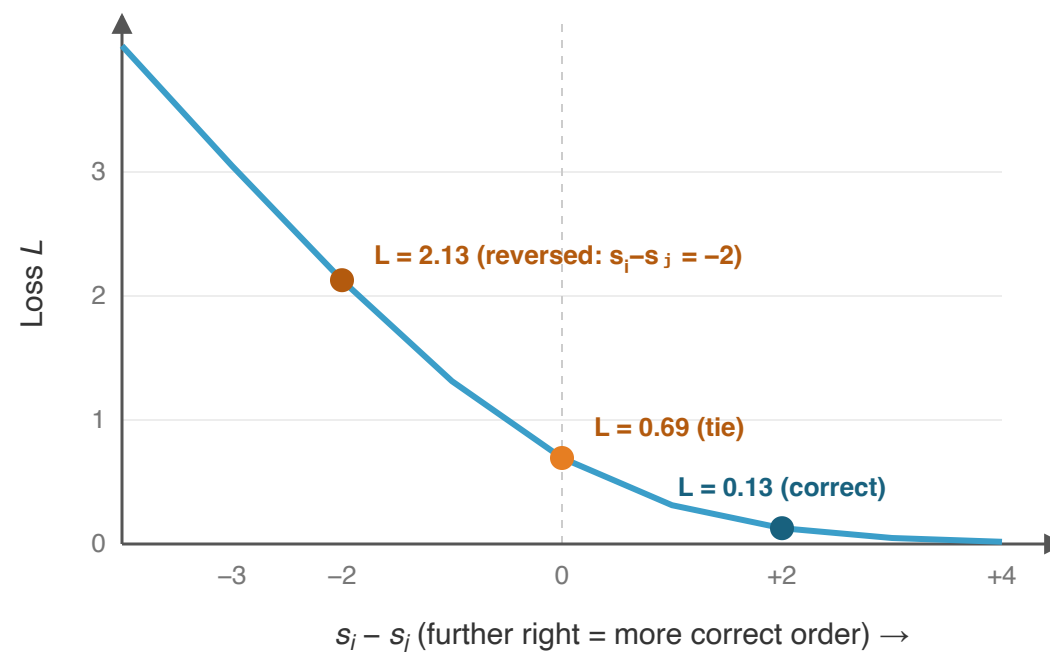
Case 2: Incorrect order ($s_i = 1, s_j = 3$)

$$\begin{aligned} L &= \log(1 + e^{-1 \cdot (1-3)}) = \log(1 + e^2) \\ &= \log(1 + 7.389) = \log(8.389) = 2.127 \end{aligned}$$

Large loss! -> Penalty because the order is reversed

Case 3: Equal scores ($s_i = 2, s_j = 2$)

$$L = \log(1 + e^0) = \log(2) = 0.693$$



Reversed order is penalized heavily; correct order costs almost nothing

Burges et al., "Learning to Rank using Gradient Descent", ICML, 2005

Loss Function: Measures gap between predictions and ground truth. Smaller = better predictions

Gradient = Derivative of loss w.r.t. parameters. Indicates **direction and magnitude** of parameter updates

Gradient Descent: Update parameters opposite to the gradient:

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial L}{\partial \theta}$$

η : learning rate, $\frac{\partial L}{\partial \theta}$: gradient of loss

Training Loop Flow

1. Prediction: Model outputs scores



2. Loss computation: Gap between prediction and ground truth



3. Gradient computation -> Parameter update

Repeat 1-3 until convergence

Derive the gradient of the loss $L_{ij} = \log(1 + e^{-\alpha(s_i - s_j)})$ with respect to score s_i . This determines the direction of model parameter updates.

Step 1: Let $u = -\alpha(s_i - s_j)$, then $L = \log(1 + e^u)$

$$\frac{\partial L}{\partial u} = \frac{e^u}{1 + e^u} = \frac{1}{1 + e^{-u}} \quad (\text{sigmoid function})$$

Step 2: Apply the chain rule (for the case $\bar{P}_{ij} = 1$)

$$\begin{aligned} \frac{\partial L}{\partial s_i} &= \frac{\partial L}{\partial u} \cdot \frac{\partial u}{\partial s_i} = \frac{e^{-\alpha(s_i - s_j)}}{1 + e^{-\alpha(s_i - s_j)}} \cdot (-\alpha) \\ &= -\alpha \cdot \frac{1}{1 + e^{\alpha(s_i - s_j)}} \end{aligned}$$

RankNet gradient (when d_i is more relevant than d_j)

$$\boxed{\frac{\partial L_{ij}}{\partial s_i} = -\frac{\alpha}{1 + e^{\alpha(s_i - s_j)}}} \quad (\text{negative value} \rightarrow \text{direction that increases } s_i)$$

$s_i \ll s_j$ (large error) $\rightarrow$ Gradient $\approx -\alpha$ (large correction)

$s_i \gg s_j$ (correct order) $\rightarrow$ Gradient ≈ 0 (no correction needed)

Gradient intuition

Verify the behavior of gradient $\lambda_{ij} = \frac{\partial L_{ij}}{\partial s_i} = \frac{-\alpha}{1+e^{\alpha(s_i-s_j)}} (\alpha = 1)$:

$s_i - s_j$	Gradient λ_{ij}	Interpretation
-4	-0.982	Large error -> Strong correction
-2	-0.881	Error -> Correction
0	-0.500	Even odds -> Moderate correction
+2	-0.119	Correct -> Weak correction
+4	-0.018	Sufficiently correct -> Almost no correction

The gradient is **always negative** -> It acts in the direction of increasing s_i (= pushing d_i higher in the ranking). Conversely, $\frac{\partial L}{\partial s_j} = +\lambda_{ij}$ decreases s_j .

Limitations of RankNet

Problem: All pairs are treated equally

RankNet computes the same form of loss for every pair (d_i, d_j) . However, in ranking:

Ordering mistakes at the **top** are severe (users actually see them)

Ordering mistakes at the **bottom** are minor (users do not see them)

Example: Swapping ranks 1 and 2 vs. swapping ranks 99 and 100

From an nDCG perspective, the former is far more serious

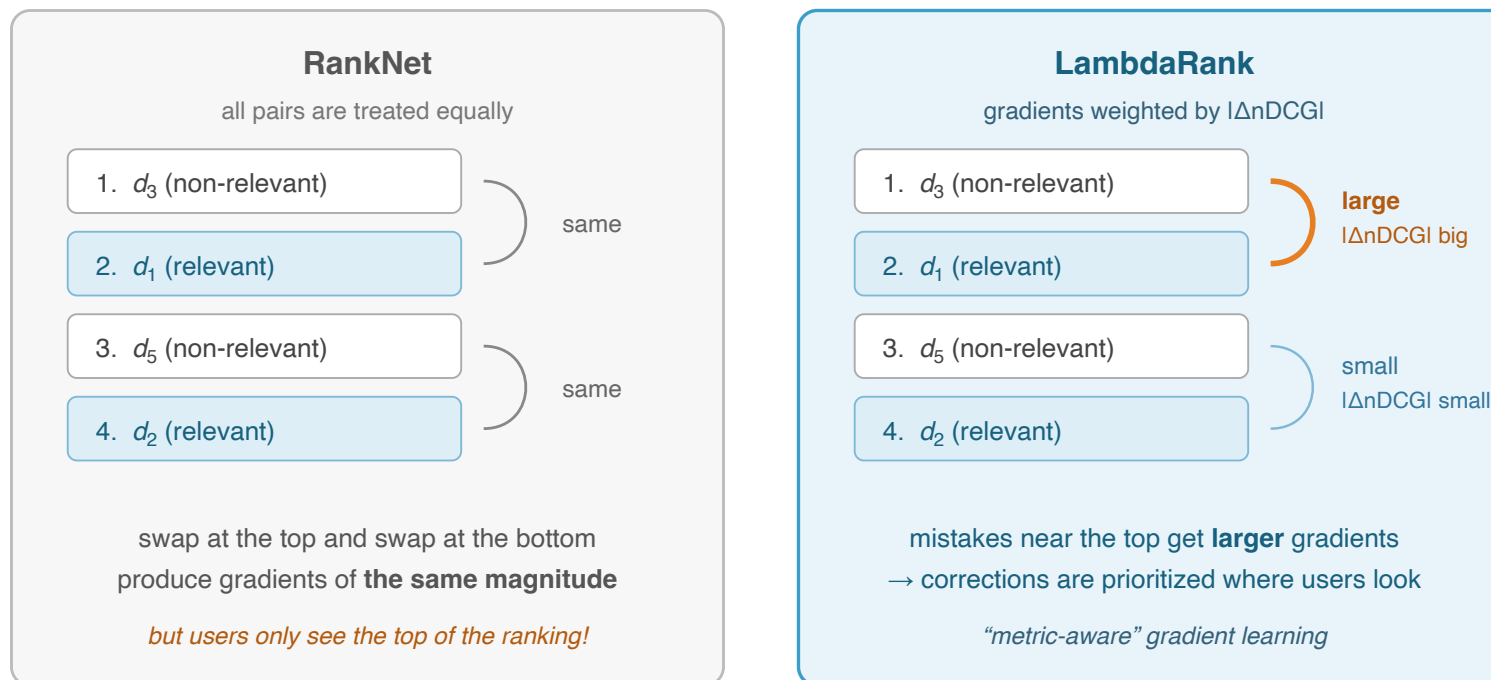
But RankNet assigns the same weight of loss to both pairs

-> **LambdaRank** solves this problem

Problem: nDCG emphasizes top positions but is **non-differentiable** w.r.t. scores (positions are discrete).

LambdaRank [Burges et al., 2006]: Instead of defining a loss, **directly define the gradient** --- multiply

RankNet's gradient by the nDCG change when swapping the pair: $\lambda_{ij} = \frac{-\alpha}{1+e^{\alpha(s_i-s_j)}} \times |\Delta \text{nDCG}_{ij}|$



LambdaMART is a method that optimizes LambdaRank gradients using **gradient boosted decision trees**.

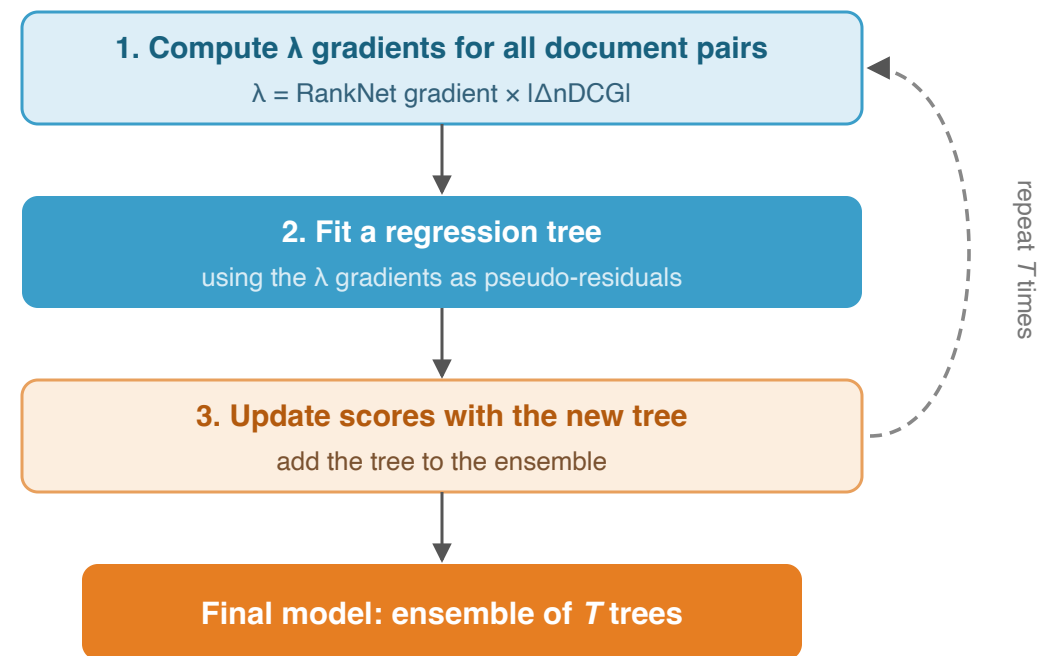
Lambda: LambdaRank's Lambda gradients

MART: Gradient boosted decision trees.

Sequentially adds trees using Lambda gradients as pseudo-residuals

Why tree models? --- They capture nonlinear interactions, are robust to sparse features, and are fast to train and infer

Track record: Adopted by top teams in the Yahoo! LTR Challenge (2010). Also used in Microsoft Bing's search ranking



Aspect	Pointwise	Pairwise	Listwise
Input unit	Individual document	Document pair	Entire document list
Loss function	Regression / classification loss	Pairwise ordering loss (cross-entropy, etc.)	List-level loss (likelihood, etc.)
Optimization target	Absolute score accuracy	Number of correctly ordered pairs	Ranking metric (nDCG, etc.)
Representative methods	Linear regression, SVM regression	RankNet, RankSVM	ListNet, AdaRank
Strengths	Simple, easy to implement	Directly learns ordering	Optimizes for the final metric
Weaknesses	Does not directly optimize ordering	Treats top and bottom equally	High computational cost

LambdaMART is a hybrid: By combining a Pairwise loss function (RankNet) with Listwise weights ($|\Delta \text{nDCG}|$), it integrates the advantages of both. The most successful LTR method in practice

Note: The above three-category classification is based on how the loss function is defined, and the boundaries are not strict. Methods like LambdaMART that span categories also exist

Learning to Rank is not limited to academic research; it has been deployed in major search services worldwide.

Google

Uses hundreds of features for ranking. Introduced RankBrain (neural-based LTR) in 2015, affecting 15% of search queries (a widely known industry fact)

Microsoft Bing

Adopted LambdaMART from the start. RankNet/LambdaRank were technologies developed at Microsoft Research [Burgess, MSR-TR-2010-82]

Yahoo!

The Yahoo! Learning to Rank Challenge (2010) galvanized interest in the research community [Chapelle and Chang, 2011]

Beyond Web Search

LTR is also used by Amazon (product search), LinkedIn (people search), Indeed (job search), and more (widely known industry applications)

Important: LTR differs from Transformer-based document ranking. LTR takes **numerous sparse hand-crafted features** as input, whereas neural ranking takes **the text itself** as input. Modern search engines combine both

Significance of LTR

Integration of multiple signals: Combines BM25, PageRank, CTR, etc. in a single model

Theoretical foundation: Pointwise → Pairwise → Listwise, culminating in LambdaMART

Practicality: Core technology in commercial search engines since the early 2010s

Limitations of LTR

Hand-crafted features: Requires domain knowledge and extensive experimentation

Lack of text understanding: Cannot solve vocabulary mismatch through features

Scalability: Requires redesign for each new domain

BM25 (single scoring function)

Term frequency and document length only



Learning to Rank

Integration of hand-crafted features



Neural IR (next topic)

Automatic feature learning from text

The promise of deep learning: Instead of hand-crafted features, **automatically learn** representations from the text itself

Deep Learning and Information Retrieval

Deep learning provided solutions to two fundamental problems in information retrieval. This was a qualitative leap that could not be achieved within the Learning to Rank framework.

Before: Limitations of Conventional Methods

Exact match dependence: Cannot match when query and document do not share the same terms (vocabulary mismatch problem)

Hand-crafted features: Requires manual design and selection of hundreds of features

Discrete representations: Terms are represented as one-hot vectors, unable to capture semantic relationships between terms

After: Contributions of Deep Learning

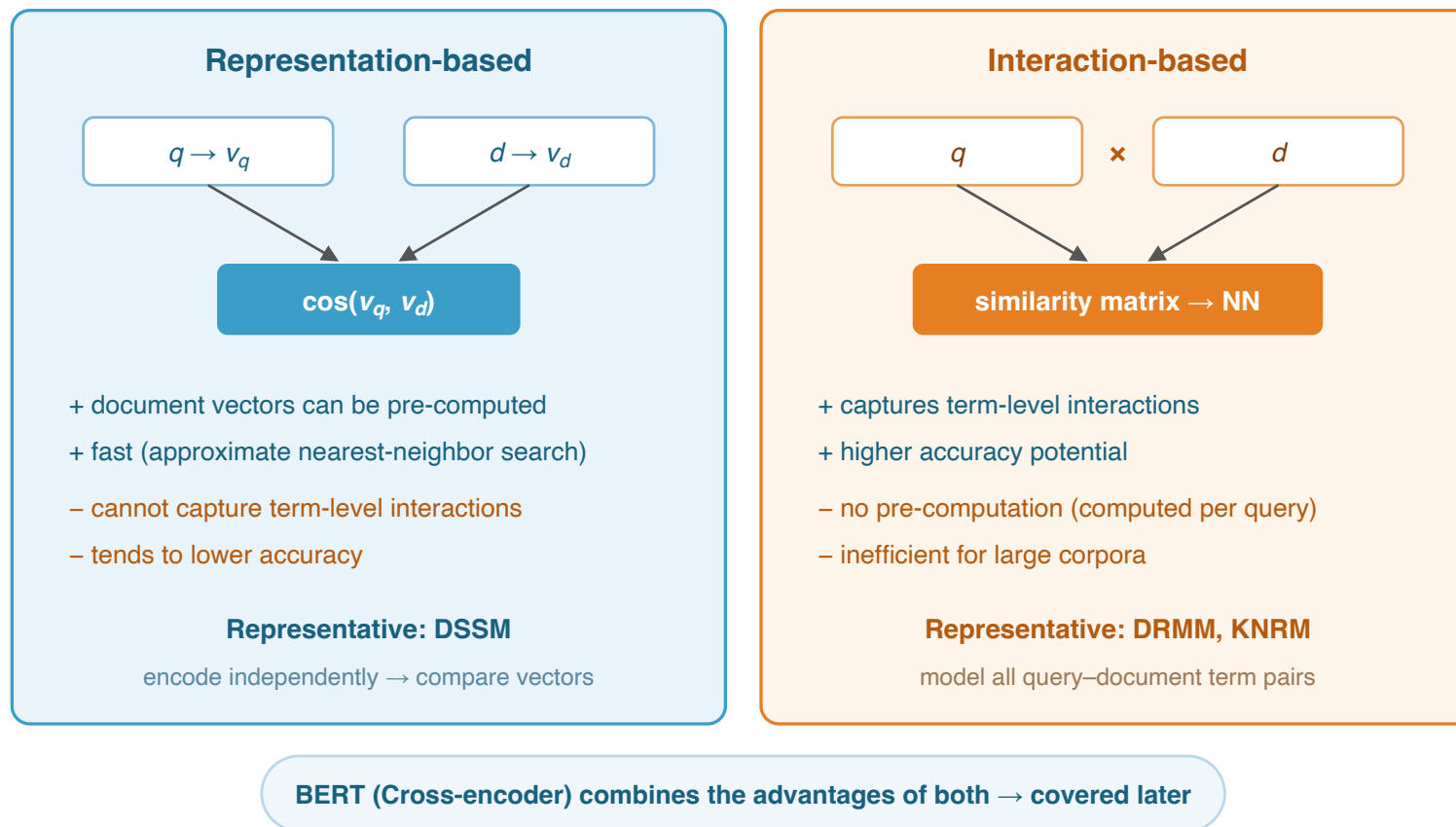
Continuous vector representations:

Represent terms, sentences, and documents as dense vectors, capturing semantic similarity as distance

Automatic feature extraction: Neural networks learn useful features directly from raw data

Flexible matching: Synonyms like "automobile" and "car" are placed close together in vector space

Pre-BERT neural ranking models fall into **two approaches** by how they compute the query-document relationship --- a fundamental tradeoff that shaped later model designs.



Representation-based Model

An approach that encodes the query and document **independently** into fixed-length vectors and computes ranking scores from the similarity between vectors

Query $q \rightarrow$ Encoder $f_q \rightarrow$ Vector $\mathbf{v}_q$

Document $d \rightarrow$ Encoder $f_d \rightarrow$ Vector $\mathbf{v}_d$

Score: $s(q, d) = \cos(\mathbf{v}_q, \mathbf{v}_d)$

Key advantage: Since document vectors $\mathbf{v}_d$ are independent of the query, they can be **pre-computed offline**. At search time, only similarity computation with the query vector is needed

Query: "Kyoto autumn leaves"

Encoder f_q

Neural network

$\mathbf{v}_q = [0.23, -0.41, \dots, 0.15]$

Documents are encoded similarly.

Score computed as $\cos(\mathbf{v}_q, \mathbf{v}_d)$

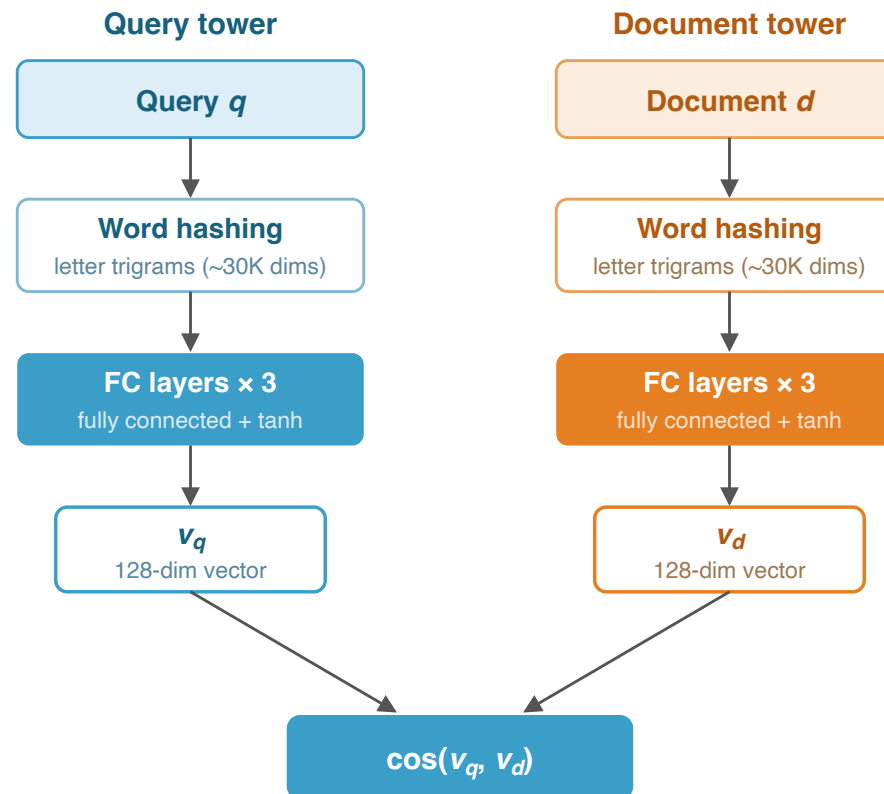
DSSM [Huang et al., 2013] is a pioneering representation-based model. Key innovation: **Word Hashing using letter trigrams**, solving vocabulary explosion and unknown word problems.

Word Hashing: "good" $\rightarrow$ "#good#" $\rightarrow$ trigrams {#go, goo, ood, od#}

Original vocabulary: ~500K dimensions

Trigrams: ~30K dimensions (**16x reduction**, collision rate: 0.0044%)

Training: Bing click-through data (click = positive, random doc = negative)



*q and d encoded independently $\rightarrow$ no term-level interactions
(Huang et al., CIKM 2013)*

Interaction-based Model

An approach that explicitly models **term-level interactions** between each query term and each document term, and computes ranking scores from these interaction patterns

Processing flow:

1. **Build a similarity matrix:** Compute cosine similarity between embedding vectors for all pairs of query terms q_i and document terms d_j

$$M[i, j] = \cos(\text{embed}(q_i), \text{embed}(d_j))$$

2. **Feature extraction:** Extract relevance signals from the similarity matrix (CNN, kernels, etc.)
3. **Scoring:** Process extracted features through a feed-forward network to produce the final score

Similarity Matrix: Query "Kyoto fall tourism" × Document terms

	Kyoto	autumn	spots	guide
Kyoto	1.00	0.12	0.35	0.08
fall	0.15	0.62	0.10	0.05
tourism	0.41	0.09	0.55	0.48

High values = semantically related. Captures non-matching pairs like "fall"-"autumn"

DRMM (Deep Relevance Matching Model) [Guo et al., 2016] is a representative interaction-based model that uses **histograms** of cosine similarities between query and document terms as features.

Basic idea:

1. For each query term q_i , compute cosine similarity with all document terms
2. Convert the similarity distribution into a **fixed-bin** histogram
3. Feed each histogram into a feed-forward network
-> matching score
4. Weight by query term importance (gating mechanism) and aggregate

Histogram Bin Design

Bin	Range	Captured relationship
1	$[-1, -0.5)$	Unrelated terms
2	$[-0.5, 0)$	Weak relation
3	$[0, 0.5)$	Moderate relation
4	$[0.5, 1.0)$	Strong semantic relation
5	$\{1.0\}$	Exact match (dedicated bin)

Key point: By having a dedicated exact match bin $\{1.0\}$, the model can capture both lexical and semantic matching

Feed-forward network:

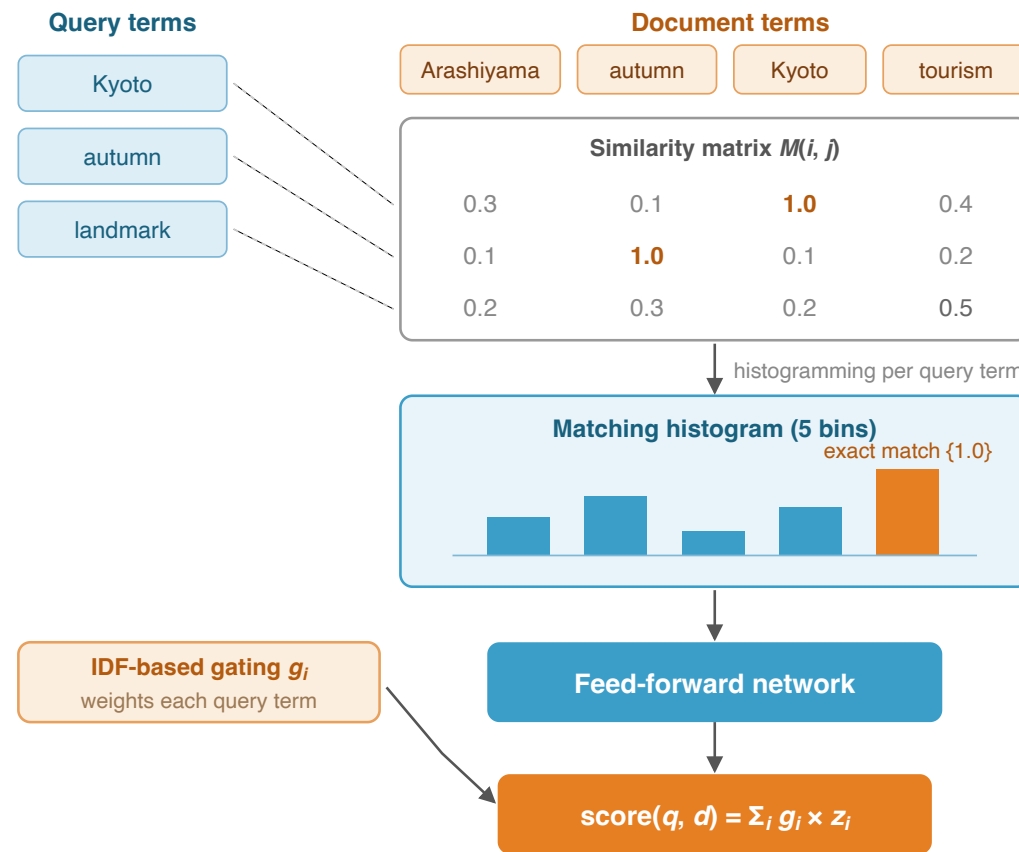
Takes each query term's histogram (5-dimensional vector) as input and outputs a matching score z_i .

Parameters are shared across all query terms

IDF-based gating mechanism:

Computes the importance g_i of each query term based on IDF, and weights the matching scores. Suppresses the influence of common terms ("the", "is") and emphasizes content terms ("Kyoto", "autumn leaves")

$$\text{score}(q, d) = \sum_i g_i \times z_i$$



A dedicated exact-match bin {1.0} lets DRMM capture both lexical and semantic matching

Both approaches have clear strengths and weaknesses, and are used depending on the purpose. Hybrid models have also been proposed.

Aspect	Representation-based	Interaction-based
Efficiency	Excellent: Document vectors can be pre-computed	Fair: Query-dependent computation required every time
Accuracy (pre-BERT)	Fair: Relatively lower	Excellent: Fine-grained term-level matching
Vocabulary mismatch handling	Good: Proximity in vector space	Excellent: Explicitly captured via similarity matrix
Deployment	Can serve as first-stage retrieval	Used as a reranker
Input independence	Query and document are independent	Query and document processed jointly

Essence of the tradeoff: Representation-based models are efficient because they process query and document independently, but miss fine-grained term-level interactions. Interaction-based models achieve higher accuracy by considering all term pairs but require more computation

Hybrid model: DUET [Mitra et al., 2017] combines representation-based and interaction-based components to leverage the advantages of both

Expectations for neural ranking models were high, but before BERT's arrival, it was **difficult to surpass conventional keyword matching methods**. This period left an important lesson about the importance of data.

Meta-analysis by Yang et al. [2019b]

Under limited data conditions, most neural ranking methods **could not outperform BM25 + query expansion** baselines

Lin [2018] raised the question: "Are neural ranking models truly superior to conventional keyword matching techniques?"

The root cause was that **training data available to academic researchers was limited**

In environments without access to large-scale data such as click logs from commercial search engines, it was difficult to demonstrate the superiority of neural models

Important lessons

1. **Model complexity < Data scale and quality:** No matter how sophisticated the architecture, it cannot perform without sufficient training data
2. **Importance of strong baselines:** BM25 + query expansion is extremely strong. A qualitative leap is needed to surpass it
3. **Fairness of evaluation:** Weak baseline implementations lead to overestimation of improvement

What broke through this limitation was the combination of the large-scale pretrained model **BERT** and the large-scale training dataset **MS MARCO**

The Arrival of BERT

Nogueira and Cho [2019] applied BERT to document ranking with dramatic improvements. The approach is simple.

Input (Cross-encoder):

$[CLS] q_1 q_2 \dots [SEP] d_1 d_2 \dots [SEP]$

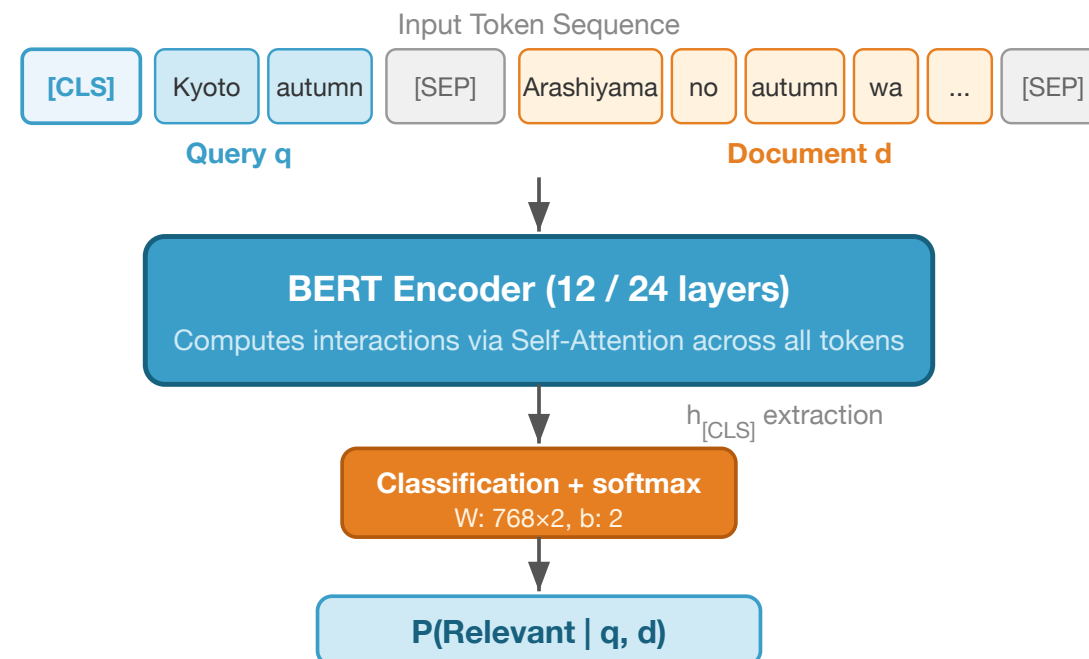
Query and document **concatenated** and fed to BERT

All tokens **attend to each other** across all layers

Scoring:

Extract $[CLS]$ output: $\mathbf{h}_{CLS}$

Linear layer: $s = \mathbf{w}^T \mathbf{h}_{CLS} + b$ (binary cross-entropy loss)



Cross-encoder: Captures deep interactions but cannot pre-compute document representations

BERT (Bidirectional Encoder Representations from Transformers) [Devlin et al., 2019] is a **pretrained language model** that has acquired general knowledge of language from massive amounts of text.

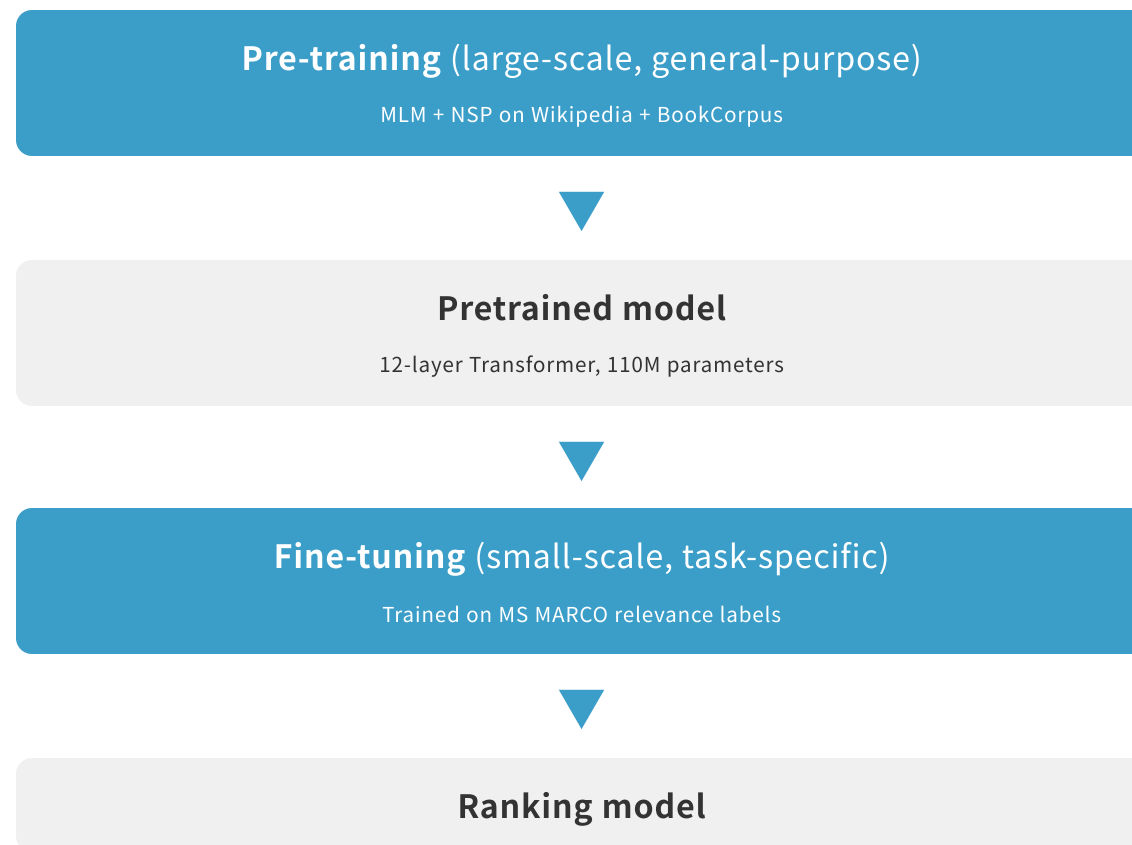
Pre-training:

Trained on two tasks using massive text (Wikipedia + BookCorpus, approximately 3.3 billion words):

Masked Language Model (MLM): Masks 15% of input tokens and recovers them from surrounding context. Acquires bidirectional contextual understanding

Next Sentence Prediction (NSP): Predicts whether two sentences are consecutive. Acquires understanding of inter-sentence relationships

BERT's Learning Paradigm



Fine-tuning:

Using the general linguistic knowledge acquired through pre-training as a foundation, specialize with **small amounts of task-specific data**. For ranking, the only additional parameters are a linear layer

Significance for IR:

Linguistic knowledge acquired through pre-training (word meanings, grammar, inter-sentence relationships) transfers to ranking

Enables building high-accuracy models with **small amounts of task data**

Enables flexible matching based on context for the vocabulary mismatch problem

Difference from prior neural models: Pre-BERT models trained only on task data, requiring large amounts of training data. BERT acquires linguistic knowledge from massive text in advance and can adapt with small amounts of data

Pre-train & Fine-tune Paradigm

Acquire general knowledge from large-scale data, then specialize with small amounts of task-specific data. This two-stage learning is the key to BERT's success

BERT's input is composed of the **element-wise sum** of three types of embeddings. Segment Embeddings play a crucial role in distinguishing between query and document.

Roles of the three embeddings:

Token Embeddings: Embeddings for each token split by the WordPiece tokenizer. Subword units from a vocabulary of approximately 30,000

Segment Embeddings: Embeddings that distinguish Segment A (= query) from Segment B (= document). Allows BERT to recognize which tokens come from the query and which from the document

Position Embeddings: Embeddings that provide positional information for tokens. Since Transformers have no inherent notion of position, these serve as the cue for word order

Structure of the Input Representation

Input	[CLS]	Kyoto	autumn	[SEP]	Kyoto's	autumn	...	[SEP]
Segment	A (Query)				B (Document)			
Position	0	1	2	3	4	5	...	n

Maximum 512 token limit

BERT's Position Embeddings support a maximum of 512 tokens. Long documents must be split (e.g., split into paragraphs and take the maximum score)

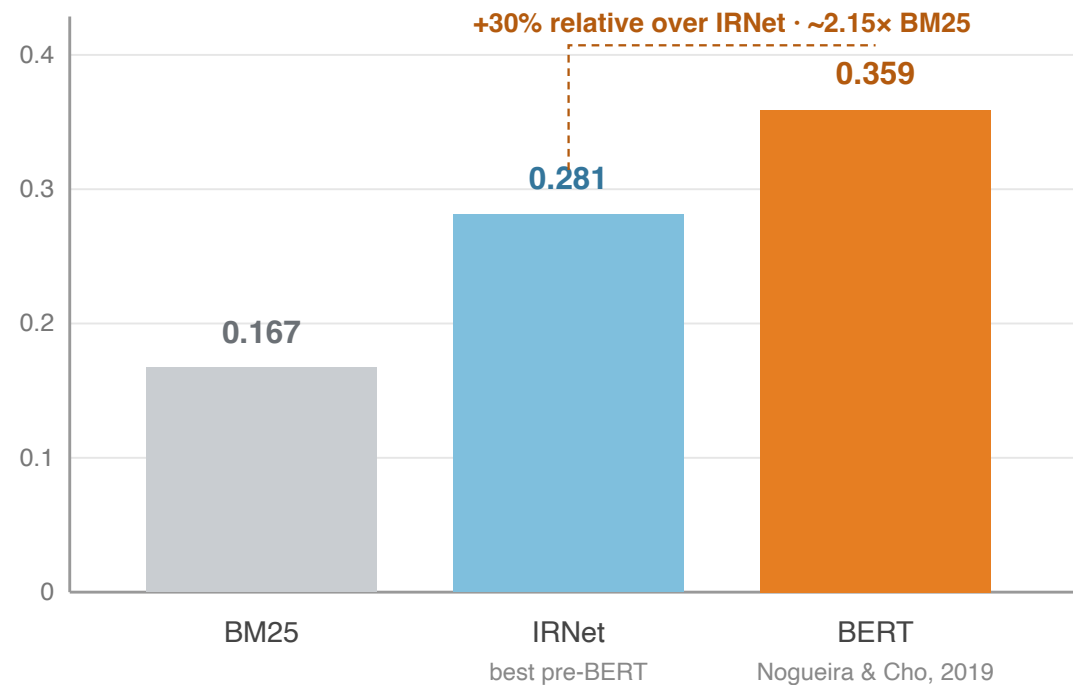
BERT brought a dramatic improvement to the MS MARCO passage ranking leaderboard.

Method	MRR@10
BM25 (Baseline)	0.167
IRNet (CNN/IR)	0.281
BERT [Nogueira & Cho]	0.359

~8-point absolute improvement over IRNet

~30% relative improvement (0.281 → 0.359)

~2.15x over BM25 baseline



MS MARCO leaderboard, January 2019

MS MARCO Leaderboard, as of January 2019; Nogueira and Cho, "Passage Re-ranking with BERT", arXiv, 2019

BERT's academic results were adopted by the world's largest commercial search engines within just a few months.

Google (October 2019):

"BERT is the **biggest leap forward in search in the past 5 years**, and one of the biggest leaps in the history of search"

Affected **1 in 10** English search queries

Particularly improved results for long natural language queries

Microsoft Bing (November 2019):

"Using large-scale Transformer models to deliver the **largest quality improvement ever** to Bing customers"

Deployed BERT-based reranking worldwide

Five Factors Behind BERT's Success

1. **Pre-training:** Acquired general linguistic knowledge from massive text (Wikipedia + BookCorpus)
2. **Fine-tuning:** Efficiently adapted with small amounts of task-specific data. Only a linear layer added for ranking
3. **Bidirectional context understanding:** Considers context from both left and right (GPT is left-to-right only)
4. **Availability of MS MARCO:** 8.8M passages, ~500K training queries. Large-scale training data became widely available
5. **Simple application method:** Applicable to ranking with minimal changes - -- just [CLS] + linear layer

Lin [2018]'s skepticism was retracted in Lin [2019]. The combination of BERT + MS MARCO broke through the data shortage barrier

1. **Learning to Rank** = Supervised learning based on hand-crafted features. **LambdaMART**, a hybrid of Pointwise/Pairwise/Listwise, achieved widespread success in both practice and benchmarks
2. **Neural ranking models** = Two families: representation-based (DSSM: independent vectorization) and **interaction-based** (DRMM: similarity matrix -> histogram)
3. Pre-BERT neural models struggled to surpass BM25 due to data scarcity
4. **The arrival of BERT** = The pre-training + fine-tuning paradigm. Achieved ~30% relative improvement on MS MARCO and was adopted by Google and Bing

Next time: How do we measure "search quality"?:

What is **Relevance**?

Evaluation metrics: Precision, Recall, AP, nDCG

Test collections: TREC, MS MARCO

How to create qrels (relevance judgments)

To determine whether BM25 or BERT is "better," a fair evaluation framework is essential. Lecture 3 covers the methodology for quantitatively measuring search quality